Machine Coding: File Explorer

- **Problem:** Build a file explorer (like the VS Code sidebar): show a nested tree of folders and files, expand and collapse folders, create files and folders, rename and delete them, and move around with the keyboard.
- **Tech stack:** React 19 + TypeScript (Vite, as set up in the task folder), plain CSS
- **Time limit:** 60 minutes

PART A: Thought process

The same steps work for any machine coding task:

1. **Understand the requirements:** functional, non-functional, clarifying questions, assumptions.
2. **List interactions / use cases:** every action and its expected result. This is your checklist.
3. **Design the architecture:** components, helpers, services and who owns what.
4. **Design the state:** the smallest set of state; derive the rest.
5. **Design data and contracts:** example data, API shape, component props.
6. **Trace the data flow:** one interaction end to end before writing code.
7. **Find the hard parts:** the 2-4 things that break naive versions.
8. **Plan the build order:** smallest working version first, then one layer at a time.
9. **Cover states and edge cases:** initial, loading, success, empty, error, plus problem-specific ones.
10. **Fix the file structure.**
11. **Write the code.**
12. **Wrap up:** what is handled, trade-offs, what you would add next.

Time budget for 60 minutes

Phase	Time
Requirements + clarifying questions	5 min
Use cases, architecture, state, contracts	7 min
Build: mock API, normalize, render rows, expand/collapse	12 min
Build: create + rename with the inline input and validation	12 min
Build: delete (subtree), selection, status path	7 min
Build: keyboard navigation + focus	9 min
Loading / error / empty states, styling, walkthrough	8 min

Think out loud. Say what you are deciding and why ("I'll keep the tree as a flat map by id, not nested objects, because rename and delete then touch one or two entries instead of rebuilding every parent on the path"). The interviewer grades your reasoning as much as the result, and talking lets them steer you early.

PART B: Apply it to this problem

1. Understand the requirements

Functional requirements

- Load the tree from an API (mocked) and show a loading state, then the tree or an error with Retry.
- Show folders and files with indentation by depth; folders show an open/closed chevron and icon.
- Click a folder to expand or collapse it. The root starts expanded.
- Inside a folder, folders come first, then files, both sorted A-Z (`file2` before `file10`).
- **Select** a row by clicking; show the full path of the selected item (e.g. `/my-project/src/App.tsx`).
- **Create** a file or folder inside the selected folder (or next to the selected file) with an inline name input.
- **Rename** the selected item inline (toolbar button or `F2`).
- **Delete** the selected item; deleting a folder deletes everything inside it (ask for confirmation).
- **Validate names:** not empty, no `/` or `\`, not `.` or `..`, and unique inside the same folder.
- **Keyboard:** Up/Down move, Right opens a folder (or goes to its first child), Left closes a folder (or goes to the parent), Home/End, Enter toggles, F2 renames, Delete deletes, inside the input Enter saves and Escape cancels.
- An expanded empty folder shows "Empty folder".

Non-functional requirements

- **Performance:** expand, rename and delete must stay fast with thousands of nodes. Updates should touch O(1) entries, and only visible rows are rendered.
- **Scalability:** the data model must allow virtualization and lazy loading of folder contents later.
- **Error handling:** API failure shows an error with Retry; invalid names never reach the tree.
- **Accessibility:** ARIA tree pattern (`role="tree"`, `treeitem`, `aria-level`, `aria-expanded`, `aria-selected`), one tab stop (roving `tabIndex`), visible focus ring, errors announced with `role="alert"`.
- **Reusability:** pure tree helpers (no React) that can be unit tested; `NameInput` reused for create and rename.
- **Extensibility:** easy to add drag-and-drop move, search, context menu or persistence without changing the model.

Clarifying questions to ask the interviewer

1. Does the data come from an API, and does it come as a nested tree or a flat list? Do changes need to be saved to the server?
2. Which operations are needed: only expand/collapse, or also create, rename, delete? What about move (drag and drop)?
3. What are the naming rules: are duplicates allowed, is the check case-sensitive, which characters are forbidden?
4. How big can the tree get (hundreds or hundreds of thousands of nodes)? Should folders load lazily?
5. Is keyboard navigation and screen-reader support expected?
6. Should the expanded state survive a page reload?

Assumptions we will build with

- The API returns the whole tree once, nested (`GET /api/tree`), mocked with a 600 ms delay. Changes stay on the client.
- Operations: expand/collapse, select, create file/folder, rename, delete. No move or drag-and-drop.
- Names are unique per folder, **case-insensitive** (like Windows/macOS). A file and a folder cannot share a name.
- One root folder that cannot be deleted, but can be renamed.
- Toolbar actions work on the selected item: new items go into the selected folder, or next to the selected file.
- Up to a few thousand nodes; virtualization is a "more time" item.
- No persistence on reload.

---

2. Identify interactions / use cases

Action / input	**Expected behavior / output**
App opens	"Loading files..." then the tree with the root expanded and selected
API fails	"Could not load files" + Retry; Retry shows loading again and refetches
Click a folder	Selects it and toggles open/closed; chevron and icon change
Click a file	Selects it; footer shows its full path
Expand an empty folder	Shows an "Empty folder" line under it
+ File / **+ Folder**	Opens the selected folder (or the file's parent), shows an input row under it, focused
Type an existing name (any case)	Red border + "already exists in this folder" shown while typing
Enter with an invalid name	Input stays open, error shown (also for an untouched empty input)
Enter with a valid name	Item created, sorted in place, selected and focused
Escape in the input	Input closes, nothing changes, focus goes back to the selected row
Click away (blur) with a valid / invalid name	Valid: saved. Invalid: cancelled
Rename or `F2`	Name becomes an input with the base name selected (`Button` of `Button.tsx`)
Rename to the same name	Treated as cancel, no update
Delete or `Delete` key on a file	Confirm, then removed; parent selected
Delete on a folder	Confirm "and the N items inside it", then the whole subtree is removed
Delete with the root selected	Button disabled; key does nothing
`ArrowDown` / `ArrowUp`	Selection and focus move to the next / previous visible row
`ArrowRight`	Closed folder: opens. Open folder: goes to first child. File: nothing
`ArrowLeft`	Open folder: closes. Otherwise: goes to the parent
`Home` / `End`	First / last visible row

Action / input	Expected behavior / output
`Enter` on a folder	Toggles it
`Tab` into the tree	Lands on the selected row only (one tab stop)

This table is the implementation checklist and the test plan.

---

3. Design the architecture

Module	Responsibility
`api/fileApi.ts`	Mock `fetchTree()`: returns nested JSON after a delay, or fails with `?fail`
`utils/tree.ts`	Pure helpers: `normalize`, `flattenVisible`, `collectSubtree`, `getPath`, `validateName`, `treeReducer`
`App.tsx`	Fetches the tree; shows loading / error / explorer
`components/FileExplorer.tsx`	Owns the tree, expanded set, selection and draft; toolbar, keyboard and focus
`components/TreeRow.tsx`	Draws one row: indent, chevron, icon, name (or the rename input)
`components/NameInput.tsx`	Inline input for create and rename: validation, Enter / Escape / blur

```
App                        (fetch + loading/error)
└── FileExplorer           (state: tree, expanded, selectedId, draft)
    ├── toolbar buttons    (+ File, + Folder, Rename, Delete)
    ├── <ul role="tree">   (keyboard handler)
    │   ├── TreeRow  x N   (one per visible row)
    │   │   └── NameInput  (only while renaming this row)
    │   └── draft row      (NameInput while creating, under its parent)
    └── footer             (path of the selected item)

utils/tree.ts  <- used by App (normalize) and FileExplorer (everything else)
```

Start simple: the first version is one component that renders the rows. `NameInput` is split out once create and rename need the same input logic; `TreeRow` once the row markup grows ARIA attributes.

Why flat rows and not a recursive `<Folder>` component? Keyboard navigation needs "the next visible row", which is just `rows[i + 1]` in a flat list. A flat list is also what virtualization needs. The recursive version is fine for a 20-minute version; we mention it as the trade-off.

---

4. Design the state / data model

State or field	Purpose
`tree.nodes`	Flat map `id -> { id, name, type, parentId, childIds }`

State or field	Purpose
`tree.rootId`	Where to start rendering
`expanded: Set<string>`	Ids of open folders (UI state, kept out of the nodes)
`selectedId: string`	The selected row; also the only row with `tabIndex=0`
`draft: Draft | null`	The open inline input: `{ kind: 'create', parentId, type }` or `{ kind: 'rename', id }`
`rows` (derived)	Visible rows `{ id, depth }`, computed from `nodes` + `expanded` every render
`targetFolderId` (derived)	Selected folder, or the selected file's parent

```
export interface FsNode {
  id: string
  name: string
  type: 'file' | 'folder'
  parentId: string | null
  childIds: string[]
}

export interface Tree {
  rootId: string
  nodes: Record<string, FsNode>
}

export type Draft =
  | { kind: 'create'; parentId: string; type: NodeType }
  | { kind: 'rename'; id: string }

const [tree, dispatch] = useReducer(treeReducer, initialTree)
const [expanded, setExpanded] = useState(() => new Set([initialTree.rootId]))
const [selectedId, setSelectedId] = useState(initialTree.rootId)
const [draft, setDraft] = useState<Draft | null>(null)
```

Senior notes

- **Derived, not stored:** visible rows, depth, sort order, the selected path and "is empty folder" are all computed. Storing `depth` or a sorted order would go stale after every rename or create.
- `expanded` **lives outside the nodes:** open/closed is view state, not file data. The server never needs it, and a second view of the same tree could have different folders open.
- **Ownership:** `App` owns loading/error; `FileExplorer` owns the tree and all UI state; `NameInput` owns only the text being typed. Children get data and callbacks.
- **Only one** `draft`**:** a discriminated union makes "creating and renaming at the same time" impossible.
- `useReducer` **for the tree:** create / rename / delete are three named actions in one pure function that can be unit tested without React.

---

5. Design the data structures and contracts

What the API sends (nested, easy for humans):

```
{
  "id": "root", "name": "my-project", "type": "folder",
  "children": [
    { "id": "src", "name": "src", "type": "folder", "children": [
      { "id": "app", "name": "App.tsx", "type": "file" }
    ] },
    { "id": "docs", "name": "docs", "type": "folder", "children": [] },
    { "id": "readme", "name": "README.md", "type": "file" }
  ]
}
```

What we keep (flat, easy for updates):

```
{
  rootId: 'root',
  nodes: {
    root:   { id: 'root',   name: 'my-project', type: 'folder', parentId: null,   childIds: ['src',
'docs', 'readme'] },
    src:    { id: 'src',    name: 'src',        type: 'folder', parentId: 'root', childIds: ['app'] },
    app:    { id: 'app',    name: 'App.tsx',    type: 'file',   parentId: 'src',  childIds: [] },
    docs:   { id: 'docs',   name: 'docs',       type: 'folder', parentId: 'root', childIds: [] },
    readme: { id: 'readme', name: 'README.md',  type: 'file',   parentId: 'root', childIds: [] },
  },
}
```

Why stable ids? Names change (rename) and repeat across folders (`index.ts` in many places), so a name or path cannot be the key. New nodes get `crypto.randomUUID()`. Ids are the React `key`, the map key and the selection value.

Why `parentId`? Delete needs to update the parent; Left arrow and the footer path walk up. Without it we would search the whole tree.

API contract (mock)

Call	Input	Output
`fetchTree()`	nothing	`Promise<ApiNode>` (nested tree) or rejects with `Error('Could not load files')`

With a real backend, mutations would be `POST /api/nodes { parentId, name, type } -> { id }`, `PATCH /api/nodes/:id { name }`, `DELETE /api/nodes/:id`.

Component contracts

```
<FileExplorer initialTree={tree} />

<TreeRow node depth isExpanded isSelected onClick={(node) => ...} ref={...}>
  {/* optional rename input */}
</TreeRow>

<NameInput
  initialName="Button.tsx"
  validate={(name) => string | null}   // error message or null
  onCommit={(name) => ...}             // trimmed, valid, changed
  onCancel={() => ...}
/>
```

---

6. Trace the data flow

Create a file inside `src`:

```
User clicks the "src" row
  -> select('src'): selectedId = 'src'; toggle('src'): expanded += 'src'
  -> render: flattenVisible() now includes the children of src
User clicks "+ File"
  -> targetFolderId = 'src' (selected is a folder)
  -> expanded += 'src' (already there), draft = { kind: 'create', parentId: 'src', type: 'file' }
  -> render: a draft row with <NameInput initialName=""> appears under the src row
  -> NameInput effect: input.focus()
User types "app.TSX"
  -> NameInput value = 'app.TSX'; validate() -> '"app.TSX" already exists in this folder'
  -> red border + role="alert" message (no state in the parent changes)
User types "utils.ts" and presses Enter
  -> validate() -> null -> finish(true) -> onCommit('utils.ts')
  -> FileExplorer.commitDraft: id = crypto.randomUUID()
       dispatch({ type: 'create', id, parentId: 'src', nodeType: 'file', name: 'utils.ts' })
       select(id): selectedId = id, pendingFocus = id
       draft = null
  -> treeReducer: new node + src.childIds = [...src.childIds, id]   (2 entries change)
  -> render: draft row gone, utils.ts row appears in sorted position, aria-selected=true
  -> effect after render: rowRefs.get(id).focus()
  -> footer: /my-project/src/utils.ts
```

---

7. Important technical challenges

Challenge 1: Updating a deeply nested tree immutably

Problem: with nested objects, renaming `src/components/Button.tsx` means copying `Button`, then `components.children`, then `components`, then `src.children`, then `src`, then the root. Every update is a recursive search plus a copy of the whole path. Delete and "find parent" are also searches.

Solution: normalize once after loading. Each node is in a flat map with `parentId` and `childIds`. Rename touches 1 entry, create touches 2, delete removes the subtree and touches the parent. The map is copied shallowly (`{ ...nodes }`), and unchanged nodes keep their identity.

```
export function normalize(root: ApiNode): Tree {
  const nodes: Tree['nodes'] = {}
  const stack = [{ node: root, parentId: null as string | null }]
  while (stack.length) {
    const { node, parentId } = stack.pop()!
    const children = node.children ?? []
    nodes[node.id] = { id: node.id, name: node.name, type: node.type, parentId, childIds:
children.map((c) => c.id) }
    children.forEach((c) => stack.push({ node: c, parentId: node.id }))
  }
  return { rootId: root.id, nodes }
}

case 'rename':
  return { ...tree, nodes: { ...nodes, [action.id]: { ...nodes[action.id], name: action.name } } }
```

Challenge 2: Rendering only visible rows, sorted, in the right order

Problem: we need a list of what is on screen right now: only children of open folders, folders before files, natural sort, with a depth for indentation. Keyboard navigation also needs "the next visible row".

Solution: derive `rows` every render with an iterative depth-first walk that only goes into expanded folders. Push children in reverse so the first one is popped first. An explicit stack also avoids stack overflow on very deep trees (tested with 20,000 levels).

```
export function flattenVisible(nodes: Tree['nodes'], rootId: string, expanded: Set<string>): Row[] {
  const rows: Row[] = []
  const stack: Row[] = [{ id: rootId, depth: 0 }]
  while (stack.length) {
    const row = stack.pop()!
    rows.push(row)
    if (!expanded.has(row.id)) continue
    const children = sortedChildren(nodes, row.id)
    for (let i = children.length - 1; i >= 0; i--) {
      stack.push({ id: children[i].id, depth: row.depth + 1 })
    }
  }
  return rows
}

// folders first, then A-Z with numbers compared as numbers
if (a.type !== b.type) return a.type === 'folder' ? -1 : 1
return a.name.localeCompare(b.name, undefined, { numeric: true, sensitivity: 'base' })
```

Challenge 3: Delete a folder with everything inside, and clean up

Problem: deleting `src` must remove `src`, `components`, `Button.tsx` ... not just the `src` entry, or the map leaks orphan nodes. The selection might be inside the deleted subtree, an inline input might be open on it, and `expanded` still lists the deleted folders.

Solution: collect the subtree ids (iterative), delete them from the map copy, remove the id from the parent's `childIds`. In the component, drop deleted ids from `expanded`, close the draft and select the parent. The root has no parent and cannot be deleted.

```
case 'delete': {
  const node = nodes[action.id]
  if (!node?.parentId) return tree // the root cannot be deleted
  const next = { ...nodes }
  for (const id of collectSubtree(nodes, node.id)) delete next[id]
  const parent = nodes[node.parentId]
  next[parent.id] = { ...parent, childIds: parent.childIds.filter((c) => c !== node.id) }
  return { ...tree, nodes: next }
}

// FileExplorer.remove()
const removed = new Set(collectSubtree(nodes, selected.id))
if (!window.confirm(`Delete "${selected.name}" and the ${removed.size - 1} items inside it?`)) return
dispatch({ type: 'delete', id: selected.id })
setExpanded((prev) => new Set([...prev].filter((id) => !removed.has(id))))
setDraft(null)
select(selected.parentId)
```

Challenge 4: The inline name input (validation, Enter + blur, focus)

Problem: three things go wrong in a naive input.

1. **Double commit:** pressing Enter saves and unmounts the input; the browser can then fire `blur`, which saves again (two files). Escape followed by blur would save a name the user cancelled.
2. **Duplicates:** `App.tsx` and `app.tsx` in the same folder; the item's own name must not count as a duplicate while renaming.
3. **Focus:** after saving, focus must land on the new row, which does not exist in the DOM yet when the click handler runs.

Solution: a `finished` ref so the input finishes only once; `validateName` checks siblings case-insensitively and ignores the node being renamed; a `pendingFocus` ref that an effect applies after the next render.

```
// NameInput
const finished = useRef(false)
function finish(commit: boolean) {
  if (finished.current) return
  finished.current = true
  if (commit && value.trim() !== initialName) onCommit(value.trim())
  else onCancel()
}
// Enter -> finish(true) if valid; Escape -> finish(false); blur -> finish(!error)

// utils/tree.ts
const taken = nodes[parentId].childIds.some(
  (id) => id !== ignoreId && nodes[id].name.toLowerCase() === name.toLowerCase(),
)

// FileExplorer: focus a row after it renders
const pendingFocus = useRef<string | null>(null)
useEffect(() => {
  if (!pendingFocus.current) return
  rowRefs.current.get(pendingFocus.current)?.focus()
  pendingFocus.current = null
})
function select(id: string) {
  setSelectedId(id)
  pendingFocus.current = id
}
```

Keyboard navigation builds on the same pieces: `rows.findIndex(selectedId)`, then `select(rows[i + 1].id)`. The tree has one tab stop: only the selected row has `tabIndex=0`, the rest have `-1` (roving tabindex).

---

8. Implement in the correct order

#	Step	Working result
1	Types + mock `fetchTree()` + `normalize()`	Data ready, checked with `console.log`
2	`App`: loading / error / Retry	All three states visible (`?fail` for error)
3	`flattenVisible()` + render rows with indent and icons	Root's children visible
4	`expanded` Set + click to toggle	Folders open and close
5	Sorting (folders first, natural A-Z) + "Empty folder"	Order matches VS Code
6	`selectedId` + highlight + footer path	Click selects
7	`treeReducer` create + `NameInput` + toolbar buttons	Files and folders can be created
8	`validateName` + error message + Enter/Escape/blur	Invalid names blocked
9	Rename (reuse `NameInput`, F2)	Rename works, base name pre-selected
10	Delete with subtree + confirm + cleanup	Delete works, parent selected
11	Keyboard handler + roving tabindex + `pendingFocus`	Full keyboard use
12	ARIA attributes, focus ring, dark mode CSS	Accessible and polished

Stop after any step and you still have a working app.

---

9. Cover all states and edge cases

State / case	What the user sees / what happens
Initial / loading	"Loading files..." (`role="status"`)
Error	"Could not load files" + Retry (`role="alert"`)
Success	Tree with root expanded and selected
Empty folder (expanded)	"Empty folder" line under it
Creating	Input row under the target folder; folder auto-expanded
Renaming	Name replaced by an input with the base name selected
Invalid name while typing	Red border + message; not shown for an untouched empty create input
Enter on an empty name	"Name is required" shown, input stays open
Spaces around the name	Trimmed before validating and saving
Duplicate in a different case	Blocked (`app.TSX` vs `App.tsx`)
Same name in another folder	Allowed (uniqueness is per folder)
Rename to the same name	No update, treated as cancel

State / case	What the user sees / what happens
Name with `/` or `\`, or `.` / `..`	Blocked
Enter then blur / Escape then blur	Only the first one counts (`finished` ref)
Blur with an invalid name	Cancelled, nothing saved
Create while a file is selected	Created in the file's parent folder
Delete root	Button disabled, key ignored
Delete a folder containing the selection	Parent of the deleted folder selected
Deleted folders in `expanded`	Removed from the set
Confirm cancelled	Nothing changes
`ArrowRight` on an open empty folder / a file	Nothing
`ArrowUp` on the first row / `ArrowDown` on the last	Nothing
Keys typed in the name input	Ignored by the tree handler (no arrow navigation while typing)
Very deep tree	Iterative walks, no recursion limit
Long names	Ellipsis; full path in the footer
Unmount during fetch (StrictMode)	`ignore` flag drops the late response

---

10. Final structure

```
_27_file_explorer/
└── src/
    ├── types.ts                ApiNode, FsNode, Tree, Row, Draft, TreeAction
    ├── api/
    │   └── fileApi.ts          mock fetchTree()
    ├── utils/
    │   └── tree.ts             pure helpers + treeReducer
    ├── components/
    │   ├── FileExplorer.tsx    state, toolbar, keyboard, focus
    │   ├── TreeRow.tsx         one row
    │   └── NameInput.tsx       create / rename input
    ├── App.tsx                 loading / error / explorer
    ├── App.css                 explorer styles
    ├── index.css               theme tokens (light + dark)
    └── main.tsx                unchanged from the Vite template
```

---

11. Full working code

`src/types.ts`

```ts
export type NodeType = 'file' | 'folder'

// Shape the server sends: nested, easy to read
export interface ApiNode {
  id: string
  name: string
  type: NodeType
  children?: ApiNode[]
}

// Shape we keep in state: flat map, easy to update
export interface FsNode {
  id: string
  name: string
  type: NodeType
  parentId: string | null
  childIds: string[]
}

export interface Tree {
  rootId: string
  nodes: Record<string, FsNode>
}

// One visible line of the tree
export interface Row {
  id: string
  depth: number
}

// The inline name input that is open right now
export type Draft =
  | { kind: 'create'; parentId: string; type: NodeType }
  | { kind: 'rename'; id: string }

export type TreeAction =
  | { type: 'create'; id: string; parentId: string; nodeType: NodeType; name: string }
  | { type: 'rename'; id: string; name: string }
  | { type: 'delete'; id: string }
```

src/api/fileApi.ts

```
import type { ApiNode } from '../types'

const MOCK_TREE: ApiNode = {
  id: 'root',
  name: 'my-project',
  type: 'folder',
  children: [
    {
      id: 'src',
      name: 'src',
      type: 'folder',
      children: [
        {
          id: 'components',
          name: 'components',
          type: 'folder',
          children: [
            { id: 'button', name: 'Button.tsx', type: 'file' },
            { id: 'modal', name: 'Modal.tsx', type: 'file' },
          ],
        },
        { id: 'app', name: 'App.tsx', type: 'file' },
        { id: 'main', name: 'main.tsx', type: 'file' },
      ],
    },
    {
      id: 'public',
      name: 'public',
      type: 'folder',
      children: [{ id: 'favicon', name: 'favicon.svg', type: 'file' }],
    },
    { id: 'docs', name: 'docs', type: 'folder', children: [] },
    { id: 'package', name: 'package.json', type: 'file' },
    { id: 'readme', name: 'README.md', type: 'file' },
  ],
}

// Mock GET /api/tree. Open the app with ?fail to see the error state.
export function fetchTree(): Promise<ApiNode> {
  const shouldFail = new URLSearchParams(window.location.search).has('fail')
  return new Promise((resolve, reject) => {
    setTimeout(() => {
      if (shouldFail) reject(new Error('Could not load files'))
      else resolve(structuredClone(MOCK_TREE))
    }, 600)
```

```
  })
}
```

`src/utils/tree.ts`

```ts
import type { ApiNode, FsNode, Row, Tree, TreeAction } from '../types'

// Nested server JSON -> flat map keyed by id
export function normalize(root: ApiNode): Tree {
  const nodes: Tree['nodes'] = {}
  const stack: { node: ApiNode; parentId: string | null }[] = [{ node: root, parentId: null }]
  while (stack.length) {
    const { node, parentId } = stack.pop()!
    const children = node.children ?? []
    nodes[node.id] = {
      id: node.id,
      name: node.name,
      type: node.type,
      parentId,
      childIds: children.map((c) => c.id),
    }
    children.forEach((c) => stack.push({ node: c, parentId: node.id }))
  }
  return { rootId: root.id, nodes }
}

// Folders first, then names A-Z (file2 before file10)
function compareNodes(a: FsNode, b: FsNode): number {
  if (a.type !== b.type) return a.type === 'folder' ? -1 : 1
  return a.name.localeCompare(b.name, undefined, { numeric: true, sensitivity: 'base' })
}

export function sortedChildren(nodes: Tree['nodes'], id: string): FsNode[] {
  return nodes[id].childIds.map((c) => nodes[c]).sort(compareNodes)
}

// Only the rows the user can see: a node's children are visited only if it is expanded
export function flattenVisible(nodes: Tree['nodes'], rootId: string, expanded: Set<string>): Row[] {
  const rows: Row[] = []
  const stack: Row[] = [{ id: rootId, depth: 0 }]
  while (stack.length) {
    const row = stack.pop()!
    rows.push(row)
    if (!expanded.has(row.id)) continue
    const children = sortedChildren(nodes, row.id)
    // push in reverse so the first child is popped first
    for (let i = children.length - 1; i >= 0; i--) {
      stack.push({ id: children[i].id, depth: row.depth + 1 })
    }
  }
  return rows
}
```

```
// The node and everything below it
export function collectSubtree(nodes: Tree['nodes'], id: string): string[] {
  const ids: string[] = []
  const stack = [id]
  while (stack.length) {
    const current = stack.pop()!
    ids.push(current)
    stack.push(...nodes[current].childIds)
  }
  return ids
}

export function getPath(nodes: Tree['nodes'], id: string): string {
  const parts: string[] = []
  for (let n: FsNode | undefined = nodes[id]; n; n = n.parentId ? nodes[n.parentId] : undefined) {
    parts.unshift(n.name)
  }
  return '/' + parts.join('/')
}

// Returns an error message, or null if the name is fine
export function validateName(
  nodes: Tree['nodes'],
  parentId: string | null,
  rawName: string,
  ignoreId?: string,
): string | null {
  const name = rawName.trim()
  if (!name) return 'Name is required'
  if (/[/\\]/.test(name)) return 'Name cannot contain / or \\'
  if (name === '.' || name === '..') return 'This name is reserved'
  if (parentId === null) return null
  const taken = nodes[parentId].childIds.some(
    (id) => id !== ignoreId && nodes[id].name.toLowerCase() === name.toLowerCase(),
  )
  return taken ? `"${name}" already exists in this folder` : null
}

export function treeReducer(tree: Tree, action: TreeAction): Tree {
  const { nodes } = tree
  switch (action.type) {
    case 'create': {
      const parent = nodes[action.parentId]
      const node: FsNode = {
        id: action.id,
        name: action.name,
```

```
        type: action.nodeType,
        parentId: parent.id,
        childIds: [],
      }
      return {
        ...tree,
        nodes: { ...nodes, [node.id]: node, [parent.id]: { ...parent, childIds: [...parent.childIds,
node.id] } },
      }
    }
    case 'rename':
      return { ...tree, nodes: { ...nodes, [action.id]: { ...nodes[action.id], name: action.name } } }
    case 'delete': {
      const node = nodes[action.id]
      if (!node?.parentId) return tree // the root cannot be deleted
      const next = { ...nodes }
      for (const id of collectSubtree(nodes, node.id)) delete next[id]
      const parent = nodes[node.parentId]
      next[parent.id] = { ...parent, childIds: parent.childIds.filter((c) => c !== node.id) }
      return { ...tree, nodes: next }
    }
  }
}
```

src/components/NameInput.tsx

```tsx
import { useEffect, useRef, useState } from 'react'
import type { KeyboardEvent } from 'react'

interface NameInputProps {
  initialName: string
  validate: (name: string) => string | null
  onCommit: (name: string) => void
  onCancel: () => void
}

export default function NameInput({ initialName, validate, onCommit, onCancel }: NameInputProps) {
  const [value, setValue] = useState(initialName)
  const [submitted, setSubmitted] = useState(false)
  // Enter commits, then the input unmounts and can fire blur: finish only once
  const finished = useRef(false)
  const inputRef = useRef<HTMLInputElement>(null)

  const error = validate(value)
  const showError = Boolean(error) && (submitted || value !== initialName)

  function finish(commit: boolean) {
    if (finished.current) return
    finished.current = true
    if (commit && value.trim() !== initialName) onCommit(value.trim())
    else onCancel()
  }

  function handleKeyDown(e: KeyboardEvent<HTMLInputElement>) {
    if (e.key === 'Enter') {
      if (error) setSubmitted(true)
      else finish(true)
    } else if (e.key === 'Escape') {
      finish(false)
    }
  }

  // On mount: focus and select "Button" in "Button.tsx" so typing keeps the extension
  useEffect(() => {
    const input = inputRef.current!
    const dot = input.value.lastIndexOf('.')
    input.focus()
    input.setSelectionRange(0, dot > 0 ? dot : input.value.length)
  }, [])

  return (
    <span className="name-input">
      <input
```

```
        ref={inputRef}
        value={value}
        aria-label="Name"
        aria-invalid={showError}
        aria-describedby={showError ? 'name-error' : undefined}
        onChange={(e) => setValue(e.target.value)}
        onKeyDown={handleKeyDown}
        onBlur={() => finish(!error)}
      />
      {showError && (
        <span id="name-error" role="alert" className="name-error">
          {error}
        </span>
      )}
    </span>
  )
}
```

src/components/TreeRow.tsx

```
import type { ReactNode, Ref } from 'react'
import type { FsNode } from '../types'

interface TreeRowProps {
  node: FsNode
  depth: number
  isExpanded: boolean
  isSelected: boolean
  onClick: (node: FsNode) => void
  ref: Ref<HTMLLIElement>
  children?: ReactNode // the rename input, when this row is being renamed
}

export default function TreeRow({ node, depth, isExpanded, isSelected, onClick, ref, children }:
TreeRowProps) {
  const isFolder = node.type === 'folder'
  return (
    <li
      ref={ref}
      role="treeitem"
      aria-level={depth + 1}
      aria-selected={isSelected}
      aria-expanded={isFolder ? isExpanded : undefined}
      tabIndex={isSelected ? 0 : -1}
      className={`row${isSelected ? ' selected' : ''}`}
      style={{ paddingLeft: depth * 16 + 8 }}
      onClick={() => {
        if (!children) onClick(node)
      }}
    >
      <span className="chevron" aria-hidden="true">
        {isFolder ? (isExpanded ? '▾' : '▸') : ''}
      </span>
      <span className="icon" aria-hidden="true">
        {isFolder ? (isExpanded ? '📂' : '📁') : '📄'}
      </span>
      {children ?? <span className="name">{node.name}</span>}
    </li>
  )
}
```

src/components/FileExplorer.tsx

```
import { useEffect, useReducer, useRef, useState } from 'react'
import type { KeyboardEvent } from 'react'
import type { Draft, FsNode, NodeType, Tree } from '../types'
import { collectSubtree, flattenVisible, getPath, treeReducer, validateName } from '../utils/tree'
import TreeRow from './TreeRow'
import NameInput from './NameInput'

export default function FileExplorer({ initialTree }: { initialTree: Tree }) {
  const [tree, dispatch] = useReducer(treeReducer, initialTree)
  const [expanded, setExpanded] = useState(() => new Set([initialTree.rootId]))
  const [selectedId, setSelectedId] = useState(initialTree.rootId)
  const [draft, setDraft] = useState<Draft | null>(null)

  const rowRefs = useRef(new Map<string, HTMLLIElement>())
  // Row to focus after the next render (it may not exist in the DOM yet)
  const pendingFocus = useRef<string | null>(null)

  useEffect(() => {
    if (!pendingFocus.current) return
    rowRefs.current.get(pendingFocus.current)?.focus()
    pendingFocus.current = null
  })

  const { nodes, rootId } = tree
  const rows = flattenVisible(nodes, rootId, expanded)
  const selected = nodes[selectedId]
  // New items go into the selected folder, or next to the selected file
  const targetFolderId = selected.type === 'folder' ? selected.id : selected.parentId!

  function select(id: string) {
    setSelectedId(id)
    pendingFocus.current = id
  }

  function toggle(id: string) {
    setExpanded((prev) => {
      const next = new Set(prev)
      if (next.has(id)) next.delete(id)
      else next.add(id)
      return next
    })
  }

  function startCreate(type: NodeType) {
    setExpanded((prev) => new Set(prev).add(targetFolderId))
    setDraft({ kind: 'create', parentId: targetFolderId, type })
  }
```

```
function startRename() {
  setDraft({ kind: 'rename', id: selectedId })
}

function remove() {
  if (!selected.parentId) return
  const removed = new Set(collectSubtree(nodes, selected.id))
  const message =
    removed.size > 1
      ? `Delete "${selected.name}" and the ${removed.size - 1} items inside it?`
      : `Delete "${selected.name}"?`
  if (!window.confirm(message)) return
  dispatch({ type: 'delete', id: selected.id })
  setExpanded((prev) => new Set([...prev].filter((id) => !removed.has(id))))
  setDraft(null)
  select(selected.parentId)
}

function commitDraft(name: string) {
  if (draft?.kind === 'create') {
    const id = crypto.randomUUID() // outside the reducer: reducers must stay pure
    dispatch({ type: 'create', id, parentId: draft.parentId, nodeType: draft.type, name })
    select(id)
  } else if (draft?.kind === 'rename') {
    dispatch({ type: 'rename', id: draft.id, name })
    select(draft.id)
  }
  setDraft(null)
}

function cancelDraft() {
  setDraft(null)
  select(selectedId)
}

function handleRowClick(node: FsNode) {
  select(node.id)
  if (node.type === 'folder') toggle(node.id)
}

function handleKeyDown(e: KeyboardEvent<HTMLUListElement>) {
  if (e.target instanceof HTMLInputElement) return // the name input handles its own keys
  const index = rows.findIndex((r) => r.id === selectedId)
  const isOpen = expanded.has(selectedId)
  const moveTo = (i: number) => {
    if (rows[i]) select(rows[i].id)
```

```
    }

    switch (e.key) {
      case 'ArrowDown':
        moveTo(index + 1)
        break
      case 'ArrowUp':
        moveTo(index - 1)
        break
      case 'Home':
        moveTo(0)
        break
      case 'End':
        moveTo(rows.length - 1)
        break
      case 'ArrowRight':
        if (selected.type !== 'folder') return
        if (!isOpen) toggle(selectedId)
        else if (selected.childIds.length) moveTo(index + 1)
        break
      case 'ArrowLeft':
        if (selected.type === 'folder' && isOpen) toggle(selectedId)
        else if (selected.parentId) select(selected.parentId)
        break
      case 'Enter':
        if (selected.type === 'folder') toggle(selectedId)
        break
      case 'F2':
        startRename()
        break
      case 'Delete':
        remove()
        break
      default:
        return
    }
    e.preventDefault()
  }

  function renderCreateRow(parentId: string, depth: number) {
    if (draft?.kind !== 'create' || draft.parentId !== parentId) return null
    return (
      <li
        key={`${parentId}-create`}
        role="treeitem"
        aria-level={depth + 1}
        className="row draft"
```

```
        style={{ paddingLeft: depth * 16 + 8 }}
      >
        <span className="chevron" />
        <span className="icon" aria-hidden="true">
          {draft.type === 'folder' ? '📁' : '📄'}
        </span>
        <NameInput
          initialName=""
          validate={(name) => validateName(nodes, parentId, name)}
          onCommit={commitDraft}
          onCancel={cancelDraft}
        />
      </li>
    )
  }

  return (
    <section className="explorer">
      <header className="toolbar" aria-label="File actions">
        <span className="title">Explorer</span>
        <button onClick={() => startCreate('file')} title="New file">
          + File
        </button>
        <button onClick={() => startCreate('folder')} title="New folder">
          + Folder
        </button>
        <button onClick={startRename} title="Rename (F2)">
          Rename
        </button>
        <button onClick={remove} disabled={!selected.parentId} title="Delete (Del)">
          Delete
        </button>
      </header>

      <ul role="tree" aria-label="Files" className="tree" onKeyDown={handleKeyDown}>
        {rows.map(({ id, depth }) => {
          const node = nodes[id]
          const isOpen = expanded.has(id)
          const isRenaming = draft?.kind === 'rename' && draft.id === id
          const showEmpty = node.type === 'folder' && isOpen && node.childIds.length === 0 &&
draft?.kind !== 'create'
          return [
            <TreeRow
              key={id}
              node={node}
              depth={depth}
              isExpanded={isOpen}
```

```
              isSelected={id === selectedId}
              onClick={handleRowClick}
              ref={(el) => {
                if (el) rowRefs.current.set(id, el)
                return () => {
                  rowRefs.current.delete(id)
                }
              }}
            >
              {isRenaming ? (
                <NameInput
                  initialName={node.name}
                  validate={(name) => validateName(nodes, node.parentId, name, id)}
                  onCommit={commitDraft}
                  onCancel={cancelDraft}
                />
              ) : null}
            </TreeRow>,
            renderCreateRow(id, depth + 1),
            showEmpty && (
              <li key={`${id}-empty`} role="none" className="empty" style={{ paddingLeft: (depth + 1) *
16 + 30 }}>
                Empty folder
              </li>
            ),
          ]
        })}
      </ul>

      <footer className="status" aria-live="polite">
        {getPath(nodes, selectedId)}
      </footer>
    </section>
  )
}
```

src/App.tsx

```tsx
import { useEffect, useState } from 'react'
import type { Tree } from './types'
import { fetchTree } from './api/fileApi'
import { normalize } from './utils/tree'
import FileExplorer from './components/FileExplorer'
import './App.css'

export default function App() {
  const [tree, setTree] = useState<Tree | null>(null)
  const [error, setError] = useState<string | null>(null)
  const [attempt, setAttempt] = useState(0)

  useEffect(() => {
    let ignore = false // drop the response if this effect was cleaned up
    fetchTree()
      .then((data) => {
        if (!ignore) setTree(normalize(data))
      })
      .catch((err: Error) => {
        if (!ignore) setError(err.message)
      })
    return () => {
      ignore = true
    }
  }, [attempt])

  function retry() {
    setError(null)
    setAttempt((a) => a + 1)
  }

  return (
    <main className="app">
      {error ? (
        <div className="message" role="alert">
          <p>{error}</p>
          <button onClick={retry}>Retry</button>
        </div>
      ) : !tree ? (
        <p className="message" role="status">
          Loading files...
        </p>
      ) : (
        <FileExplorer initialTree={tree} />
      )}
    </main>
```

```
  )
}
```

src/index.css

```
:root {
  font-family: system-ui, 'Segoe UI', Roboto, sans-serif;
  line-height: 1.5;
  color-scheme: light dark;
  --bg: #f5f6f8;
  --panel: #ffffff;
  --text: #1f2328;
  --muted: #6b7280;
  --border: #d0d7de;
  --hover: #eef1f4;
  --accent: #2563eb;
  --selected: #dbe7fd;
  --danger: #d1242f;
}

@media (prefers-color-scheme: dark) {
  :root {
    --bg: #0d1117;
    --panel: #161b22;
    --text: #e6edf3;
    --muted: #8b949e;
    --border: #30363d;
    --hover: #1f2630;
    --accent: #4c8dff;
    --selected: #1c3a6b;
    --danger: #ff7b72;
  }
}

* {
  box-sizing: border-box;
}

body {
  margin: 0;
  background: var(--bg);
  color: var(--text);
}
```

src/App.css

```
.app {
  max-width: 420px;
  margin: 40px auto;
  padding: 0 16px;
}

.message {
  text-align: center;
  color: var(--muted);
}

.explorer {
  background: var(--panel);
  border: 1px solid var(--border);
  border-radius: 8px;
  overflow: hidden;
}

.toolbar {
  display: flex;
  align-items: center;
  gap: 4px;
  padding: 8px;
  border-bottom: 1px solid var(--border);
}

.toolbar .title {
  flex: 1;
  font-size: 12px;
  font-weight: 600;
  text-transform: uppercase;
  letter-spacing: 0.05em;
  color: var(--muted);
}

button {
  font: inherit;
  font-size: 12px;
  padding: 4px 8px;
  border: 1px solid var(--border);
  border-radius: 6px;
  background: transparent;
  color: var(--text);
  cursor: pointer;
}

button:hover:not(:disabled) {
```

```
  background: var(--hover);
}

button:disabled {
  opacity: 0.4;
  cursor: not-allowed;
}

.tree {
  list-style: none;
  margin: 0;
  padding: 4px 0;
  max-height: 420px;
  overflow-y: auto;
}

.row {
  display: flex;
  align-items: center;
  gap: 4px;
  min-height: 28px;
  padding-right: 8px;
  font-size: 14px;
  cursor: pointer;
  user-select: none;
}

.row:hover {
  background: var(--hover);
}

.row.selected {
  background: var(--selected);
}

.row:focus-visible {
  outline: 2px solid var(--accent);
  outline-offset: -2px;
}

.chevron {
  width: 14px;
  flex-shrink: 0;
  color: var(--muted);
  text-align: center;
}
```

```
.icon {
  flex-shrink: 0;
}

.name {
  overflow: hidden;
  text-overflow: ellipsis;
  white-space: nowrap;
}

.empty {
  font-size: 13px;
  font-style: italic;
  color: var(--muted);
  min-height: 24px;
}

.name-input {
  position: relative;
  flex: 1;
}

.name-input input {
  width: 100%;
  font: inherit;
  padding: 1px 4px;
  border: 1px solid var(--accent);
  border-radius: 4px;
  background: var(--panel);
  color: var(--text);
  outline: none;
}

.name-input input[aria-invalid='true'] {
  border-color: var(--danger);
}

.name-error {
  position: absolute;
  top: 100%;
  left: 0;
  right: 0;
  z-index: 1;
  padding: 2px 6px;
  font-size: 12px;
  color: #fff;
  background: var(--danger);
```

```
  border-radius: 0 0 4px 4px;
}

.status {
  padding: 6px 10px;
  border-top: 1px solid var(--border);
  font-size: 12px;
  color: var(--muted);
  font-family: Consolas, monospace;
  overflow: hidden;
  text-overflow: ellipsis;
  white-space: nowrap;
}
```

`src/main.tsx` stays as generated by Vite. Delete `src/assets/react.svg`, it is no longer used. Run `npm run dev`; open `/?fail` to see the error state.

Self-check for the pure logic (`node --experimental-strip-types check.ts`)

```
import assert from 'node:assert/strict'
import { normalize, flattenVisible, collectSubtree, getPath, validateName, treeReducer } from
'./app/src/utils/tree.ts'

const tree = normalize({
  id: 'r', name: 'root', type: 'folder', children: [
    { id: 'b', name: 'b.txt', type: 'file' },
    { id: 's', name: 'src', type: 'folder', children: [
      { id: 'f10', name: 'file10.ts', type: 'file' },
      { id: 'f2', name: 'file2.ts', type: 'file' },
    ] },
    { id: 'a', name: 'A.txt', type: 'file' },
  ],
})
const { nodes } = tree
assert.equal(nodes.f2.parentId, 's')
assert.deepEqual(nodes.r.childIds, ['b', 's', 'a'])

// collapsed root shows only root; folders first, then numeric A-Z
assert.deepEqual(flattenVisible(nodes, 'r', new Set()), [{ id: 'r', depth: 0 }])
assert.deepEqual(flattenVisible(nodes, 'r', new Set(['r'])).map((x) => x.id), ['r', 's', 'a', 'b'])
assert.deepEqual(
  flattenVisible(nodes, 'r', new Set(['r', 's'])).map((x) => `${x.id}:${x.depth}`),
  ['r:0', 's:1', 'f2:2', 'f10:2', 'a:1', 'b:1'],
)

assert.deepEqual(collectSubtree(nodes, 's').sort(), ['f10', 'f2', 's'])
assert.equal(getPath(nodes, 'f2'), '/root/src/file2.ts')

assert.equal(validateName(nodes, 'r', '   '), 'Name is required')
assert.equal(validateName(nodes, 'r', 'a/b'), 'Name cannot contain / or \\')
assert.equal(validateName(nodes, 'r', '..'), 'This name is reserved')
assert.equal(validateName(nodes, 'r', 'SRC'), '"SRC" already exists in this folder')
assert.equal(validateName(nodes, 'r', 'src', 's'), null) // renaming itself is fine
assert.equal(validateName(nodes, 'r', 'new.ts'), null)
assert.equal(validateName(nodes, null, 'anything'), null)

let t = treeReducer(tree, { type: 'create', id: 'n', parentId: 's', nodeType: 'file', name: 'new.ts' })
assert.deepEqual(t.nodes.s.childIds, ['f10', 'f2', 'n'])
assert.equal(t.nodes.n.parentId, 's')
assert.notEqual(t.nodes, nodes) // immutable
assert.equal(nodes.n, undefined)

t = treeReducer(t, { type: 'rename', id: 'n', name: 'renamed.ts' })
assert.equal(t.nodes.n.name, 'renamed.ts')

t = treeReducer(t, { type: 'delete', id: 's' })
```

```
assert.deepEqual(Object.keys(t.nodes).sort(), ['a', 'b', 'r'])
assert.deepEqual(t.nodes.r.childIds, ['b', 'a'])
assert.equal(treeReducer(t, { type: 'delete', id: 'r' }), t) // root stays

// deep tree: no stack overflow
let deep: any = { id: 'leaf', name: 'leaf', type: 'file' }
for (let i = 0; i < 20000; i++) deep = { id: `d${i}`, name: `d${i}`, type: 'folder', children: [deep] }
const big = normalize(deep)
assert.equal(collectSubtree(big.nodes, big.rootId).length, 20001)
assert.equal(flattenVisible(big.nodes, big.rootId, new Set(Object.keys(big.nodes))).length, 20001)

console.log('self-check ok')
```

---

12. Wrap-up

Edge cases handled

- Loading, error with Retry, empty folders, late responses after unmount.
- Name rules: empty, whitespace, `/` `\`, `.` `..`, case-insensitive duplicates per folder, rename to itself.
- Enter + blur double commit, cancel on Escape, cancel on invalid blur.
- Recursive delete with cleanup of `expanded`, draft and selection; the root is protected.
- Deep trees without recursion; natural sort; long names.
- Keyboard-only use with one tab stop and focus that follows the selection.

Trade-offs to mention

- **Flat map vs nested tree:** flat makes updates O(1) and gives `parentId` for free, but needs a `normalize` step and a `collectSubtree` walk for delete. Nested is simpler to render recursively for a small demo.
- **Flat row list vs recursive components:** flat rows make keyboard navigation and virtualization easy; ARIA uses `aria-level` instead of nested `role="group"` lists. The recursive version matches the data shape more directly.
- **Sorting at render time:** always correct, costs O(k log k) per open folder per render. For huge folders, keep `childIds` sorted on insert instead.
- **`window.confirm`:** fast to build, but blocks the thread and cannot be styled; a modal or an "Undo" toast is better UX.
- **Client-only changes:** no server sync, so a reload loses changes. A real version would call the API optimistically and roll back on failure.
- **Toolbar instead of hover icons:** accessible by keyboard and less code; hover actions or a context menu are a nice addition.

What to add with more time

- **Tests:** unit tests for `tree.ts` (already sketched in the self-check), React Testing Library tests for create / rename / delete / keyboard.
- **Performance:** virtualize rows (`react-window`) for 100k nodes, keep sorted `childIds`, `React.memo` on `TreeRow` if not using the React Compiler.

- **Features:** drag-and-drop move (block moving a folder into its own descendant), search/filter that auto-expands matches, context menu, multi-select, persistence of the tree and `expanded` in `localStorage` or the server.
- **Lazy loading:** fetch a folder's children when it is first expanded, with a per-folder loading/error state.
- **Undo** for delete and rename.

---

13. Counter questions the interviewer may ask

Q1. Why normalize the tree instead of keeping the nested JSON?

Updates. With nesting, every change copies every ancestor and needs a recursive search. With a flat map, rename is one entry and create is two. We also get `parentId`, which Left arrow, delete and the path need. The cost is one O(n) `normalize` after loading.

Q2. Why is `expanded` a separate `Set` and not `isOpen` on each node?

Open/closed is view state, not file data. Keeping it out means the reducer and the server never deal with it, toggling does not copy the node map, and two views of the same tree could open different folders. A `Set` gives O(1) `has`.

Q3. Why do you derive `rows` every render instead of storing them?

Rows depend on `nodes`, `expanded` and sort order. Stored rows would need updating after every create, rename, delete and toggle, and they would drift. Deriving is O(visible rows), which is small. If it ever got slow, `useMemo` on `[nodes, expanded]` fixes it (the React Compiler already does that here).

Q4. Why `crypto.randomUUID()` in the component and not inside the reducer?

Reducers must be pure. React StrictMode calls reducers twice in development; a random id inside would create two different ids for one action. The id is created in the event handler and passed in the action.

Q5. Why not use the file path as the id?

Paths change when a parent is renamed, so every descendant's id would change, and React would remount those rows. Stable ids never change; the path is derived from `parentId` when we need it.

Q6. How does the duplicate check work while renaming?

`validateName(nodes, parentId, name, ignoreId)` compares the new name, lowercased, with the sibling names, skipping the node being renamed. So renaming `App.tsx` to `app.tsx` is allowed, but renaming it to `main.tsx` is not.

Q7. Why the `finished` ref in `NameInput`? Isn't state enough?

Enter calls `onCommit`, the parent closes the draft and the input unmounts. A `blur` that fires in the same tick still sees the old state value, so a state flag would not stop the second call. A ref updates synchronously, so the second `finish` returns early.

Q8. Why focus rows in an effect with `pendingFocus` and not right in the handler?

After create, the new row is not in the DOM yet when the handler runs; it appears only after React renders. The handler records which row should get focus, and an effect (which runs after the DOM is updated) focuses it. The same path is used for keyboard moves, so focus logic lives in one place.

Q9. Why only one `tabIndex=0` row?

That is the roving tabindex pattern for ARIA trees: Tab enters the tree once and leaves it once, instead of stopping on hundreds of rows. Arrows move inside. Only the selected row is tabbable, so Tab always returns to where the user was.

Q10. Your keyboard handler is on the `<ul>`. Why not on each row?

One handler through event bubbling instead of N. The handler needs `rows` and `selectedId`, which live in `FileExplorer` anyway. It ignores events from the name input (`e.target instanceof HTMLInputElement`) so arrows still move the cursor while typing.

Q11. What if the tree has 1 million nodes?

`normalize` is O(n) once. Rendering is the problem: render only the visible slice of `rows` with virtualization (fixed 28 px row height makes it simple). Load folder children lazily from the API instead of fetching the whole tree. Keep `childIds` sorted on insert so a render does not sort big folders.

Q12. What if the API is slow or folders load lazily?

Store a per-folder status (`idle | loading | loaded | error`) in a map. On first expand, show a "Loading..." row under the folder, fetch `GET /api/nodes/:id/children`, normalize the result into the map. Keep the `ignore` pattern (or `AbortController`) so collapsing or unmounting does not apply a stale result.

Q13. How would you save changes to a server?

Optimistic update: dispatch the change, then call the API. On failure, dispatch the reverse action (rename back, re-insert the deleted subtree we kept) and show a toast. For create, use a temporary client id and swap it for the server id when the response comes back.

Q14. How would you add drag-and-drop move?

A `move` action: remove the id from the old parent's `childIds`, add it to the new parent's, set `parentId`. Block dropping a folder into itself or a descendant: walk up from the target with `parentId` and reject if you meet the dragged id. Also run `validateName` against the new siblings.

Q15. Why `localeCompare` with `numeric: true`?

Plain string sort puts `file10` before `file2`. `numeric: true` compares digit runs as numbers, and `sensitivity: 'base'` ignores case, which matches how file managers sort.

Q16. Why iterative loops instead of recursion?

Recursion is fine for normal trees, but a very deep tree (generated or malicious data) can overflow the call stack. An explicit stack has no depth limit; the self-check runs 20,000 levels.

Q17. Why not a recursive `<Folder>` component that renders its children?

It is shorter and matches the data, and I would build it first in a 20-minute round. But "next visible row" for ArrowDown then needs a tree walk across components, and virtualization needs a flat list anyway. Flat rows with `aria-level` give both.

Q18. What happens to stale ids in `expanded` after a delete?

We remove them. They would not break rendering (deleted ids are never visited), but they would leak memory over time and could reopen a folder if an id were ever reused.

Q19. What if two users edit the same tree?

The client state can go stale. Options: refetch on focus, push updates over WebSocket and apply them as reducer actions, and have the server reject conflicting writes (version number per node) so the client can show "this item changed, reload".

Q20. Why does the "+ File" button create next to a selected file instead of being disabled?

That is how VS Code behaves, and it avoids a dead button. `targetFolderId` is derived: the selected folder, or the selected file's parent.

Q21. Is the explorer accessible to screen-reader users?

Yes: `role="tree"` / `treeitem`, `aria-level`, `aria-expanded` only on folders, `aria-selected`, a visible focus ring, icons hidden with `aria-hidden`, input errors in `role="alert"` linked with `aria-describedby`, and the path in an `aria-live` footer.

Q22. How would you test it?

Unit tests for the pure helpers: normalize, sort order, visible rows, validation, reducer immutability, recursive delete, root protection. Component tests with React Testing Library: create with Enter, duplicate shows an error, Escape cancels, delete with a mocked `confirm`, Arrow keys move `aria-selected`. One end-to-end test for the full flow.